Lab 2C: Precision & Recovery

Duration: 30-45 min
After: Context Feeding & @ Mentions slides
Continues from: Lab 2B (your Business Dashboard should have CLAUDE.md)

Goal

Master context control with @ mentions to make Claude look exactly where you want — and learn to recover when things go catastrophically wrong.

Phase 1: Feel the Cost of Imprecision

Start with a broad prompt:

```
Update the API to handle errors better.
```

Claude has to search your entire project, decide which files matter, and guess what "better" means. This burns tokens and produces generic results.

Now ask:

```
How many files did you need to read to answer that prompt?
```

Then try the same task with precision:

```
Now repeat the same analysis but only using @src/index.js
```

Claude will explain the difference. **Precision prompts reduce context cost** — which is a real production concern when you're paying per token and working in large codebases.

> ***Why this matters:*** *In a large codebase, the difference between* `@src/auth.ts` *and "look at the auth code" could be 50,000 tokens of unnecessary context.*

Phase 2: Architectural Refactor

Now use @ mentions for a real refactoring task:

```
Look at @src/index.js — the routes are all in one file.
Refactor the project to use a modular routing structure. Create:
- src/routes/metrics.js
- src/routes/tasks.js

Update src/index.js so it only contains:
- app initialization
- middleware
- route mounting

Ensure the refactor follows our CLAUDE.md conventions.
Test all endpoints afterward.
```

Watch Claude: 1. Read the specific file you pointed to 2. Identify the route groups 3. Create two new files following your CLAUDE.md conventions 4. Update the main file to import and mount them 5. Test that nothing broke

This is a real architectural improvement — not just moving code around.

Verify Nothing Broke

```
Test all the API endpoints — metrics, tasks, dashboard, stats, and health.
Show me the results.
```

All endpoints should return valid data. The refactor was invisible to the API consumer.

Phase 3: Disaster Recovery

This is how developers actually use Claude Code — things break, and you fix them.

Simulate a bad merge:

```
Simulate a bad merge: delete src/routes/metrics.js and remove
the metrics route import and mounting from src/index.js
```

Your metrics API is now completely gone. Now recover:

```
Metrics functionality disappeared after a bad merge.
Reconstruct the metrics routes based on:
- the API specification from Lab 2A
- the patterns in @src/routes/tasks.js
- our CLAUDE.md conventions
Restore the import and mounting in src/index.js.
Ensure all metric endpoints work again.
```

Watch Claude: 1. Read the remaining code to understand the pattern 2. Check CLAUDE.md for conventions 3. Rebuild the missing file to match the existing architecture 4. Restore the wiring in the main file 5. Test that everything works

What Just Happened?

Claude didn't just regenerate a file — it **reconstructed missing functionality** by analyzing what was still there. It used `tasks.js` as an architectural reference, followed CLAUDE.md conventions, and verified the result.

In the real world, you'll use this pattern when: - A merge conflict corrupts a file - You accidentally delete something - A dependency upgrade breaks existing code - You need to reverse-engineer a missing component

> ***The pattern:*** *Describe what's wrong + point to reference files + state what the result should look like. Claude handles the detective work.*

Phase 4: Extract Shared Patterns

Now use multi-file @ mentions to find duplication:

```
Look at @src/routes/metrics.js and @src/routes/tasks.js.
Identify duplicated patterns in validation, error responses,
or database queries. If duplication exists, extract shared helpers into:
src/utils/apiHelpers.js
Refactor both route files to use the helpers.
```

This is architecture cleanup — the kind of work that usually takes a full afternoon. Watch Claude identify the patterns, create the shared module, and refactor both files while maintaining all functionality.

Phase 5: Add a Feature Using Multiple File References

```
Look at @src/routes/tasks.js and @public/index.html — add the ability
to create a new task from the frontend UI. Add a simple form at the top
of the tasks section with fields for title, priority (dropdown), and
assigned_to. When submitted, POST to the API and refresh the task list.
```

Open `http://localhost:3100` in your browser. You should see a form above the task list. Create a task and verify it appears.

Phase 6: Clean Commits

```
Commit these changes as separate commits:
one for the refactor, one for the shared helpers, one for the new feature
```

Watch Claude organize changes into logical groups and write descriptive conventional commit messages for each.

Quick Reflection

Which pattern will you use first at work?

- □ @ mentions for targeted changes in large codebases

- □ Disaster recovery after a bad merge or accidental delete
- □ Multi-file refactoring with architectural cleanup
- □ Extracting shared patterns across files

Go Deeper

URL @ mentions — bring in external docs:

```
@https://expressjs.com/en/guide/error-handling.html Update our error
handling middleware to match Express best practices from this guide
```

Claude will fetch the Express documentation, read the patterns, and update your code.

Stress test the recovery:

Delete TWO files at once:

```
Delete src/routes/metrics.js and src/routes/tasks.js
```

Then:

```
Both route files were deleted. Restore them both based on the API spec,
the shared helpers in @src/utils/apiHelpers.js, and CLAUDE.md conventions.
Test everything.
```

□Lab 2C Checkpoint

- [ ] Routes split into separate modules (src/routes/)
- [ ] Recovery from simulated bad merge successful
- [ ] Shared helpers extracted into src/utils/
- [ ] All API endpoints still work
- [ ] Frontend has a "New Task" form that creates tasks
- [ ] Clean git commits (refactor + helpers + feature)

> ***Keep Claude running. Return to slides when your instructor is ready.***